# Group Project: Build a Backend That Solves a Real Problem

Object-Oriented Programming | Zambia University College of Technology

| | |
|---|---|
| Group size | 5 students. You choose your own team. |
| Total marks | 100 |
| Proposal due | Monday, 26 October 2026 |
| Final submission due | Wednesday, 4 November 2026 (video link and repository link) |
| Interviews | From Monday, 9 November 2026. Each group gets a time slot. |

## The idea

Think of this like a hackathon. You will not be given a problem. **You find one.** Pick a real problem that real people in Zambia face. Then build software that solves it.

You will use everything we have learned in this course: classes, objects, encapsulation, inheritance, polymorphism, interfaces, exceptions and collections. You will use **Javalin** to turn your Java code into a backend that other programs can talk to. Then you will add a simple **front end** so a person can actually use it.

The goal is not a big project. The goal is a **well-designed** one that every member of your group can explain.

## What you must deliver

| Part | What it is | When |
|---|---|---|
| 1. Proposal | One page: the problem, who uses it, and your main classes. | Mon 26 Oct |
| 2. Working project | Your Java and Javalin code on GitHub, with a simple front end. | Wed 4 Nov |
| 3. Demo video | A YouTube video, 3 minutes maximum. | Wed 4 Nov |
| 4. Interview | Each member is questioned about the project, one by one. | From Mon 9 Nov |
| 5. Peer rating | Each member rates the other four, privately. | Wed 4 Nov |

## How to choose a problem

Look around you. Where do people waste time, lose records, or struggle to find information? Good problems are small and clear. Examples of the *kind* of problem (do not simply copy these):

- A queue or booking system for a small business
- A tracker for something a community or a church group manages by hand
- A tool that helps farmers, traders or students keep records

A good test: **can you say who will use it and what it does for them in one sentence?** If not, the problem is too vague.

> **Avoid the default.** Many groups will think of a generic "student management system". If your idea is that common, you must make it clearly different, or pick another problem. Your proposal will be rejected if it is too generic or too big to finish.

## Part 1: The Proposal (5 marks)

Submit **one page** by Monday, 26 October. It must answer:

- **The problem.** What is going wrong, and for whom?
- **The users.** Who will use your system?
- **Your main classes.** List 4 to 6 classes and what each one is responsible for.
- **Your team.** Names and student numbers of all 5 members.

I will reply with **Approved** or **Change this**. You should not start building until you are approved. This saves you from wasting a week on an idea that will not work.

## Part 2: The Working Project (25 marks)

Your problem is free. The technical requirements are **fixed**. Your project must have all of the following:

| Requirement | What it means |
|---|---|
| Real classes | Classes that model your problem, not one giant class doing everything. |
| Encapsulation | Private fields, with controlled access through methods. |
| Inheritance or interface | At least one parent-child or interface relationship that you actually use. |
| Polymorphism | At least one place where the same call behaves differently for different objects. |
| Custom exceptions | At least one exception class you wrote, thrown and handled properly. |
| Collections | Lists or maps to hold your objects. |
| Javalin endpoints | At least 5 REST endpoints, including GET, POST, and one of PUT or DELETE. |
| Saving data | Data must survive a restart. A file or SQLite is fine. |
| A front end | A simple page (plain HTML and JavaScript is fine) served by Javalin that uses your endpoints. |

**The front end does not need to be pretty.** It needs to work. Marks go to your design and your code, not to colours.

### Your GitHub repository

- Create one repository for the group. Every member must commit code from their own account.
- Include a short README: the problem, how to run the project, and a list of your classes.
- I will look at the commit history. It shows who did what.

## Part 3: The Demo Video (10 marks)

Make a video of **3 minutes maximum**. Upload it to YouTube as **Unlisted** and submit the link. Anything past 3 minutes will not be marked.

Pitch it like a hackathon team. Your video should show:

- The problem, in the first 30 seconds
- Your solution actually running, not slides
- One or two OOP ideas from your code, explained simply

## Part 4: The Interview (50 marks)

This is the biggest part of your grade. Each member is interviewed **individually**, about 15 minutes each, with the project open on screen.

I will ask you things like:

- "Show me where polymorphism happens in your code. What would break if I removed it?"
- "Why is this field private?"

- "Add a new field to this class now. What else needs to change?"
- "Which part did you write? Walk me through it."

Every member gets a **different** part of the project to explain, and may be asked to make a small change live. You get your own mark. A strong group does not rescue a weak interviewee.

> **A word about AI tools.** You may use AI to learn and to help you write code. But you must understand every line your group submits. If you cannot explain your own code in the interview, you will lose marks, no matter how good the project looks. Use AI as a tutor, not as a replacement for your own thinking.

## Part 5: Peer Rating (10 marks)

Each member privately rates the other four on contribution: effort, reliability and quality of work. I read these alongside the GitHub history. A member who did not contribute can lose these marks and more.

## Marks summary

| Part | Marks | Marked as |
|---|---|---|
| Proposal | 5 | Group |
| Working project and code quality | 25 | Group |
| Demo video | 10 | Group |
| Individual interview | 50 | Individual |
| Peer contribution rating | 10 | Individual |
| **Total** | **100** | |

## Timeline

| Date | What happens |
|---|---|
| Wed 21 Oct | Assignment released. Form your group of 5. |
| Mon 26 Oct | One-page proposal due. I reply with Approved or Change this. |
| Wed 4 Nov | Repository link, video link and peer ratings due. |
| From Mon 9 Nov | Individual interviews. The slot list will be published in advance. |

## Before you submit: checklist

[ ] My proposal was approved.
[ ] The project runs from the README instructions.
[ ] All 5 members have commits in the repository.
[ ] All requirements in the table above are in the code.
[ ] The video is 3 minutes or less, set to Unlisted, and the link works.
[ ] Every member can explain every class.

Start early. Solve something real. Build less, and understand all of it.